Theoretical Computer Science, exam of 2025.06.18

SURNAME _____ NAME _____PERSONAL CODE_____
Total available time: 1h 30'. You can use books and paper notes during the exam, while electronic devices are prohibited. ***You must write the solutions on this paper alone.*** *Each correct exercise is valued 11 points.*
Multichance *students do not need to answer to Exercises 2.2 and 3.2: bar here in that case* [ ]

Exercise 1

Consider the following grammar G:
S → ABC | ε
ABC → aaX
X → ABC
A → a
BC → A

1. What type is G in the Chomsky hierarchy?
2. Use a non-grammatical formalism with minimal expressive power, among those covered in class, that characterizes the language L generated by G.

Exercise 2

1) Determine whether the following function is computable or not, justifying your answer:

$g(x, y) = \text{if } f_y(x) \neq \perp \text{ then } 1 \text{ else } 2$

2) Now consider the problem of determining whether a generic Turing machine computes the function g. Is this problem decidable? Semi-decidable? Decided?

Exercise 3

Describe a **k-tape Turing machine (TM)** and a **RAM machine** that receive as input a positive integer `n` in unary and output the sequence of numbers `1, 2, ..., n`, separated by an appropriate delimiter.

1. For the TM, output numbers in **binary**.
2. For the RAM, use `0` as a separator.
   In both cases, evaluate space and time complexities under all applicable cost criteria.

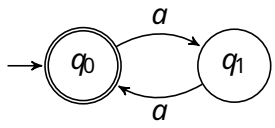
space available for solutions

Solutions

EX 1

The grammar G is **Type 0 (unrestricted)**, due to rules like `BC → A`, which is also **not monotonic**, as the right side has fewer symbols than the left.

- The language L is **regular** and can be characterized by the regular expression `(aa)*`.
  Notably, this language is **not star-free**, and therefore cannot be expressed using MFO (Monadic First-Order logic) formulas.
  Another simple characterization is via the following **deterministic finite automaton (DFA)**:



An additional characterization is via the following **regular grammar**:

```
S → aA | ε
A → aS
```

EX 2

1. The function **g is not computable**. If it were, then the Halting Problem would also be computable, which is a known undecidable problem.
The reduction from the Halting Problem to computability of g is straightforward: take an instance of the Halting Problem (machine `y` with input `x`) and return the decision value `2 - g(x, y)`. If `g` were computable, we could always solve the Halting Problem.

2. The decision problem is **decidable** (and hence also **semi-decidable**) and **decided (negatively)**. Since g is not computable, **no Turing machine computes it**, so the answer to the decision problem is always negative.

EX 3

1. **Turing Machine (TM)**:
   The TM keeps a **binary counter** on a tape and increments it once for each symbol read from the input. After each increment, it **copies** the counter's content to the output, possibly with a separator.
   1. Time complexity: `T(n) = 0(n log n)`
   2. Space complexity: `S(n) = 0(log n)`
2. **RAM Machine**:
   The procedure is similar. For each input symbol, the counter is incremented and its value printed.
   - **With constant cost model**:

    - Time: $T(n) = 0(n)$
    - Space: $S(n) = 0(1)$
- **With logarithmic cost model** (for managing the counter):
    - Time: $T(n) = 0(n \log n)$
    - Space: $S(n) = 0(\log n)$